```
371
372     def decode_payload(self, payload_bytes, source_ip=None, timestamp=None, callsign_hint=None):
373         frames, remainder = self.split_frames(payload_bytes)
374         decoded = []
375         frame_hint = callsign_hint if len(frames) <= 1 else None
376
377         for frame in frames:
378             rec = self.decode_frame(
379                 frame,
380                 source_ip=source_ip,
381                 timestamp=timestamp,
382                 callsign_hint=frame_hint,
383             )
384             if rec:
385                 decoded.append(rec)
386
387         # If the whole payload is exactly one frame but not split for some reason.
388         if not decoded and not remainder and payload_bytes and payload_bytes[0] == len(payload_bytes):
389             rec = self.decode_frame(
390                 payload_bytes,
391                 source_ip=source_ip,
392                 timestamp=timestamp,
393                 callsign_hint=callsign_hint,
394             )
395             if rec:
396                 decoded.append(rec)
397
398         return decoded
399
400     def decode_log_file(self, file_path, max_entries=300, reset_state=False):
401         entries = self.parse_log_entries(file_path, max_entries=max_entries)
402         if reset_state:
403             self.reset_state()
404
405         for raw_time, source_ip, payload, callsign_hint in entries:
406             self.decode_payload(
407                 payload,
408                 source_ip=source_ip,
409                 timestamp=raw_time,
410                 callsign_hint=callsign_hint,
411             )
412
413         return self.get_state_snapshot(stale_seconds=None)
414
415     def parse_log_entries(self, file_path, max_entries=300):
416         path = Path(file_path)
417         if not path.exists():
418             return []
419
420         text = path.read_text(encoding="utf-8", errors="ignore")
421         # Allow simulation files that include log-banner lines or mixed sections.
422         text = text.replace("PINGSTATION 3 LIVE DATA LOG\n", "")
423         full_log_pattern = re.compile(
424             r"TIME\s*:\s*(?P<time>[^\n]+)\s*.*?"
425             r"SOURCE IP\s*:\s*(?P<ip>[^\n]+)\s*.*?"
426             r"HEX DATA:\s*\n(?P<hex>[0-9a-fA-F]+)\s*.*?"
427             r"ASCII DATA:\s*\n(?P<ascii>[^\n]+)",
428             re.DOTALL,
429         )
430         full_log_hex_only_pattern = re.compile(
431             r"TIME\s*:\s*(?P<time>[^\n]+)\s*.*?"
432             r"SOURCE IP\s*:\s*(?P<ip>[^\n]+)\s*.*?"
433             r"HEX DATA:\s*\n(?P<hex>[0-9a-fA-F]+)",
434             re.DOTALL,
435         )
436         simple_pattern = re.compile(
437             r"HEX DATA:\s*\n(?P<hex>[0-9a-fA-F]+)\s*\n\s*ASCII DATA:\s*\n(?P<ascii>[^\n]+)",
438             re.DOTALL,
439         )
440
441         rows = []
442         for match in full_log_pattern.finditer(text):
443             raw_time = match.group("time").strip()
```